

PROBLEM D: THE SIMULTANEOUS STRIKE

International Computer Science Competition 2026 — Qualification Round

1. Root Cause Architecture Bug Analysis

The core structural logic flaw inside the original multiplayer battle engine pseudocode is an ordering dependencies race condition operating across discrete simulation time steps (frames). The sequence uses an immediate evaluation step where an iterative loop applies individual discrete attack elements step-by-step. Within the inner execution array loop, the engine invokes a health pool boundary rule ($hp \leq 0$) which instantly triggers an execution termination (`break`).

When two adversarial network payloads register processing actions across identical frame integers (such as the simultaneous execution tracking noted on frame 512), standard stable sorting orders place Player 1 ahead index-wise. Because Player 1's state decrement drops Player 2's health to 0 inside the immediate loop transaction pass, the termination logic breaks out of the update pipeline immediately. This prevents the corresponding damage vector owned by Player 2 from ever applying to Player 1, creating an artificial sequence bias based purely on internal array positioning.

2. Corrected Engineering Implementation

The architectural correction segregates individual state modifications into atomic frame transaction blocks. All active game events indexing on the exact same frame timestamp are grouped and resolved across current structural entities before any systemic lifepool evaluation scans check for participant expiration thresholds.

3. Python 3.9 Source Code

```
def processGame(events: list, H: int) -> list:
    if not events:
        return [H, H]

    # Enforce chronological ordering across historical event sequences
    sorted_events = sorted(events, key=lambda x: x[1])

    p1_hp = H
    p2_hp = H

    i = 0
    num_events = len(sorted_events)
```

```

while i < num_events:
    # Evaluate death termination states prior to processing subsequent frame clusters
    if p1_hp <= 0 or p2_hp <= 0:
        break

    current_frame = sorted_events[i][1]
    frame_end_index = i

    # Look ahead to aggregate all matching continuous events within the current frame
    while frame_end_index < num_events and sorted_events[frame_end_index][1] == current_frame:
        frame_end_index += 1

    # Execute concurrent calculation modifications simultaneously
    for idx in range(i, frame_end_index):
        player, _, attack = sorted_events[idx]
        if player == 1:
            p2_hp -= attack
        elif player == 2:
            p1_hp -= attack

    # Advance state pointer index to step past the current frame group
    i = frame_end_index

    # Apply lower floor normalization clamping across health metrics
    return [max(0, p1_hp), max(0, p2_hp)]

if __name__ == "__main__":
    pass

```